PROJECT DOCUMENTATION

ShopSmart E-Commerce Application

Complete technical specification, architecture overview, API guide, and deployment manual for the standalone full-stack shopping platform.

APPLICATION TYPE	ARCHITECTURE
Full-Stack Web App (React + Express)	Standalone / Database-Free (Zero XAMPP)
COURSE / LABORATORY	RELEASE VERSION
PDC50 - Full-Stack Web Development	v1.0.0 Stable

1. Executive Summary & Project Purpose

ShopSmart is an educational full-stack e-commerce web application engineered for modern web development practice. It demonstrates end-to-end integration between a responsive, single-page React frontend and an Express REST API backend.

The platform provides real-time catalog browsing, instant product search, category filtering, persistent client cart state management, checkout simulation with atomic inventory deduction, and an intelligent rule-based shopping assistant capable of parsing natural language price constraints.

> 💡 **Standalone Operation (No XAMPP or Database Server Required)**
>
> The application has been engineered to run completely standalone. It stores data in a file-backed local JSON repository (`backend/src/data/db.json`). You **do not need XAMPP, Apache, MySQL, or PostgreSQL** running on your machine.

Key System Objectives

- **Zero External Server Dependencies:** Operates 100% locally with Node.js and npm without requiring background database services.
- **Full Catalog Management:** Real-time product search, categorization, and item inventory tracking.
- **Transactional Stock Reservation:** Decrements stock upon checkout and protects against overselling.
- **AI Shopping Assistant:** Rule-based heuristic processor that extracts budget amounts (e.g., ₱2,000) and matches top catalog items.
- **Comprehensive Test Coverage:** Full automated testing across unit and integration levels with Vitest and Node Test Runner.

2. System Architecture & Technology Stack

ShopSmart adopts a decoupled client-server architecture. The frontend and backend run as independent Node.js processes managed concurrently via npm workspaces.

Layer	Technology	Key Libraries / Tools	Description
Frontend	React 19, Vite 7	React Router v7, React Context API	Single-page application with responsive layouts, route handling, and dynamic cart state.
Backend API	Node.js 22 LTS, Express 5	Helmet, Morgan, CORS, Express-Rate-Limit	RESTful API serving JSON payloads with security headers, request tracing, and logging.
Validation	Zod 4	Schema parsing & sanitization	Validates customer checkout payloads and assistant question constraints.
Data Storage	File-Backed JSON Store	Node.js File System (`fs`)	Self-contained, atomic file persistence at `backend/src/data/db.json`. Zero database server required.
Quality & Testing	Vitest & Node Test Runner	ESLint 9	Automated unit tests for cart algorithms, assistant rules, and schema validators.

3. Project Directory & File Structure

The project is organized as an npm multi-workspace repository:

```
ShopSmart/
├── backend/
│   ├── prisma/
│   │   └── seed.js            # Standalone catalog reset / seed script
│   ├── src/
│   │   ├── data/
│   │   │   └── db.json        # Local persistent data store (products & orders)
│   │   ├── lib/
│   │   │   └── prisma.js      # Database-free storage repository interface
│   │   ├── routes/
│   │   │   ├── assistant.js   # AI Assistant API route
│   │   │   ├── orders.js      # Checkout & order processing route
│   │   │   └── products.js    # Catalog retrieval & search route
│   │   ├── services/
│   │   │   └── assistant.js   # Budget parsing & heuristic recommendation logic
│   │   ├── validators/
│   │   │   └── schemas.js     # Zod input validation schemas
│   │   ├── app.js             # Express app configuration & middleware
│   │   └── server.js          # Server bootstrap & graceful shutdown
│   ├── test/                  # Backend automated unit tests
│   ├── .env                   # Environment variables (port, origin)
│   └── package.json           # Backend dependencies & scripts
├── frontend/
│   ├── src/
│   │   ├── components/        # Reusable UI components (ProductCard, StatusMessage)
│   │   ├── context/           # CartContext state provider
│   │   ├── pages/             # View pages (Products, Details, Cart, Checkout, Assistant)
│   │   ├── services/api.js    # Fetch API wrapper
│   │   ├── utils/             # Cart calculation logic & unit tests
│   │   ├── App.jsx            # Route definitions & global layout
│   │   └── main.jsx           # React root mount
│   ├── .env                   # Frontend environment configuration
│   └── package.json           # Frontend dependencies & scripts
├── docs/                      # Project documentation & reference files
├── package.json               # Root workspace configuration
└── README.md                  # Quickstart guide
```

4. Installation & Setup Guide

Running ShopSmart does not require any database setup or third-party servers. All you need is Node.js installed.

Step 1: Install Dependencies

Open PowerShell or Terminal at the project root directory (`ShopSmart`) and execute:

```
npm install
```

Step 2: Start the Application

Launch both the frontend client and the backend API concurrently using:

```
npm run dev
```

The console will display outputs from both workspaces:

- **Frontend Web Application:** `http://localhost:5173`
- **Backend REST API:** `http://localhost:3000`

Step 3: Reset or Seed Sample Data (Optional)

If you want to reset the catalog back to the original 6 products with full inventory, run:

```
npm run db:seed
```

Step 4: Run Automated Verification

To run ESLint, frontend tests, backend tests, and build check all in one command:

```
npm run check
```

5. REST API Reference

All backend endpoints are prefixed with `/api`. Responses are returned in standard JSON format.

1. Health Check

`GET` `/api/health`

Returns the operational status of the API server.

```
// Response 200 OK
{
  "status": "ok"
}
```

2. Get All Products

`GET` `/api/products`

Retrieves all active catalog products. Supports an optional query parameter `?search=<keyword>` to filter items by name.

```
// Response 200 OK
{
  "products": [
    {
      "id": 1,
      "name": "Wireless Mouse",
      "description": "Quiet wireless mouse with adjustable sensitivity and USB receiver.",
      "category": "Accessories",
      "emoji": "🖱️",
      "price": 650,
      "stock": 12,
      "active": true
    },
    ...
  ]
}
```

3. Get Product By ID

`GET` `/api/products/:id`

Retrieves a single product by its numerical ID.

- **200 OK:** Returns the product object.
- **400 Bad Request:** If `:id` is not a positive integer.

- **404 Not Found:** If the product does not exist or is inactive.

4. Create Simulated Order (Checkout)

`POST` `/api/orders`

Validates cart items, verifies stock availability, deducts inventory, and records a new order.

```
// Request Body
{
  "customerName": "Josh Manalang",
  "email": "josh@example.com",
  "address": "123 Academic Way, Angeles City",
  "items": [
    { "productId": 1, "quantity": 1 },
    { "productId": 2, "quantity": 1 }
  ]
}

// Response 201 Created
{
  "id": 2,
  "customerName": "Josh Manalang",
  "email": "josh@example.com",
  "address": "123 Academic Way, Angeles City",
  "status": "PENDING",
  "total": 2500,
  "createdAt": "2026-09-28T07:15:30.123Z",
  "items": [
    { "id": 4, "orderId": 2, "productId": 1, "quantity": 1, "unitPrice": 650 },
    { "id": 5, "orderId": 2, "productId": 2, "quantity": 1, "unitPrice": 1850 }
  ]
}
```

5. ShopSmart AI Assistant Query

`POST` `/api/assistant/questions`

Processes a user query, parses budget constraints, ranks catalog items, and outputs recommendations with traceable product sources.

```
// Request Body
{
  "question": "Which keyboard is below ₱2,000?"
}

// Response 200 OK
{
  "answer": "Based on the current ShopSmart catalog, consider: Compact Mechanical Keyboard. This sta
  "recommendedProducts": [
    { "id": 2, "name": "Compact Mechanical Keyboard", "price": 1850, "category": "Accessories" }
  ],
  "sources": [
    { "type": "product", "id": 2, "name": "Compact Mechanical Keyboard" }
  ]
}
```

6. Data Models & Schemas

Product Model

Field	Type	Description
`id`	Integer	Unique auto-incremented product identifier.
`name`	String	Product title (e.g., "Wireless Mouse").
`description`	String	Full textual description of product features.
`category`	String	Product classification (e.g., "Accessories", "Study", "Audio").
`emoji`	String	Visual emoji representation for UI cards.
`price`	Number	Retail price in Philippine Pesos (₱).
`stock`	Integer	Available inventory units. Decremented upon purchase.
`active`	Boolean	Visibility flag (default: `true`).

Order & OrderItem Models

Model	Field	Type	Description
Order	`id`	Integer	Unique order tracking number.
	`customerName`	String	Customer's full name (2–80 characters).
	`email`	String	Valid email address.
	`address`	String	Delivery address (10–300 characters).
	`status`	String	Order status (default: `"PENDING"`).
	`total`	Number	Computed order grand total in Pesos.
OrderItem	`id`	Integer	Unique line item ID.
	`orderId`	Integer	Foreign reference to the parent Order.
	`productId`	Integer	Reference to the purchased Product.
	`quantity`	Integer	Number of units ordered.

7. Quality Assurance & Troubleshooting

Automated Test Suite

- **Frontend (`vitest run`):** Tests shopping cart arithmetic, quantity increments, quantity decrements, and total calculation in `src/utils/cart.test.js`.
- **Backend (`node --test`):** Tests budget regex extraction, heuristic product ranking, traceability of AI recommendations, Zod order validation, and duplicate item rejection.

Common Troubleshooting Scenarios

Issue	Probable Cause	Solution
Port 3000 or 5173 already in use	Another background terminal or application is holding the port.	Close other terminals or stop the process in Task Manager, then run `npm run dev`.
Products not appearing	The data store was emptied or the backend is stopped.	Run `npm run db:seed` to restore all 6 sample products and restart `npm run dev`.
No XAMPP or MySQL needed?	User expected MySQL connection.	ShopSmart is completely standalone. Do not launch XAMPP or MySQL; the local JSON store handles all operations automatically.